



POLITECNICO
MILANO 1863

SCHOOL OF INDUSTRIAL AND
INFORMATION ENGINEERING

DD
Best Bike Paths

Fabio Vokrri (10738386)
Soufian Sadgal (11003465)

January 7, 2026
Version 1.0

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions, Acronyms, Abbreviations	3
1.3.1	Definitions	3
1.3.2	Acronyms	4
1.3.3	Abbreviations	4
1.4	Revision history	4
1.5	Reference Documents	4
1.6	Document Structure	5
2	Architectural Design	6
2.1	Overview	6
2.1.1	System View	6
2.1.2	Detailed View	7
2.2	Component View	9
2.2.1	Front-end Components	9
2.2.2	Application Layer Components	10
2.2.3	Data Layer	11
2.2.4	External Services	11
2.3	Deployment View	12
2.4	Runtime View	13
2.5	Component Interfaces	22
2.6	Selected Architectural Styles and Patterns	25
3	User Interface Design	27
4	Requirements Traceability	32
5	Implementation, Integration & Test Plan	37
5.1	Implementation Plan	37
5.2	Component Integration and Testing	37
5.3	System Testing	40
6	Effort Spent	42
7	References	43

List of Figures

1	High Level System View. The Event-Driven Layer resides in the Application Layer.	6
2	A more detailed view of the BBP system architecture.	9
3	Component Diagram.	12
4	System Deployment View.	12
5	User Registration Sequence Diagram.	14
6	User Login Sequence Diagram.	14

7	Update Profile Data Sequence Diagram.	15
8	Delete Account Sequence Diagram.	15
9	Browse Bike Paths Sequence Diagram.	16
10	Plan Trips Sequence Diagram.	17
11	Change Automatic Tracking Settings Sequence Diagram.	17
12	View History and Statistics of Completed Trips Sequence Diagram.	18
13	Delete Trips from History Sequence Diagram.	19
14	Manual Report Sequence Diagram.	20
15	Automatic Report Sequence Diagram.	21
16	Receive Trip Alerts Sequence Diagram.	22
17	Sign in and Sign up screens.	27
18	Home Screen and Trip Details Screen.	28
19	Map Screens and Navigation.	29
20	Report and Congratulation Screens.	30
21	Profile Screen.	31
22	Test Plan Diagram.	38

List of Tables

1	Account and Access Management Mapping.	32
2	Bike Path Discovery and Visualisation Mapping.	32
3	Trip Planning and Route Recommendation Mapping.	33
4	Trip Tracking, History and Performance Analytics Mapping.	34
5	Reporting, Validation and Publication of Path Information Mapping.	35
6	Context Enrichment and Notifications Mapping.	36
7	Effort spent in hours.	42

Listings

1 Introduction

1.1 Purpose

The *Best Bike Paths (BBP)* system has one objective in mind: support the ever-growing cycling community by providing a platform for recording, browsing, and sharing optimal bike routes. By integrating automated data acquisition from mobile sensors with manual user contributions, BBP builds a reliable inventory of safe and smooth paths. It enhances the biking experience by offering detailed performance statistics and real-time meteorological information on the go. Furthermore, BBP promotes safety by automatically identifying physical road hazards through community-verified data, which non-registered users can also benefit from. By prioritizing routes where speed limits and traffic density are bike-friendly, it promotes a safer environment for riders of all skill levels. Through its dual-mode insertion and analytical scoring, BBP becomes a comprehensive tool for both personal fitness and community navigation.

1.2 Scope

The core function of Best Bike Paths (BBP) is to collect, integrate, and provide reliable information about bike paths in order to support cyclists in making informed decisions about the routes they choose to follow. The system acts as a mediator between real-world bike path conditions, cyclists' experiences and reports, and external data sources such as meteorological services. Through BBP, users can contribute with information about bike path conditions, validate data automatically collected by the system, and explore available routes based on their suitability and current conditions. Registered users can record their trips, delete old trips, manually or automatically report obstacles or unsafe situations, confirm or correct detected information, and publish validated data to the community. Unregistered users can access publicly available information but are not allowed to contribute or modify data. The system interacts with external weather services to retrieve environmental data and enrich trip information. It also integrates with mobile devices to receive sensor data, such as GPS traces and accelerometer readings, that may indicate relevant path features or anomalies. BBP maintains a consolidated and continuously updated view of bike path conditions by merging contributions from multiple users according to their freshness, reliability, and level of confirmation.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- Software: ordered set of instructions that tells the computer how to use the available data to perform the tasks it is designed to carry out.
- Biking Trip: journey performed by bike over a period of time following a predetermined route.
- External Service: software outside the application's domain, consumed and relied upon by the primary application in order to perform the actions it is designed to execute.
- Obstacle: something, possibly dangerous, that hinders the biking trip or might block the route. Some examples of obstacles may be potholes, tree branches on the route, roadworks or road closures.
- Accelerometer: electronic sensor that measures the acceleration of the device.

- Gyroscope: electronic sensor that measures the orientation and angular velocity of the device.
- GPS (Global Positioning System): system using satellite orbiting the earth to obtain the device's location.

1.3.2 Acronyms

- BBP: Best Bike Paths.
- GPS: Global Positioning System.
- UI: User Interface.

1.3.3 Abbreviations

- G: goal.
- T: thread.
- WP: World Phenomena.
- SP: Shared Phenomena.
- TDR: Tracking Data Received.
- TDP: Tracking Data Processed.
- TC: Trip Completed.
- RP: Report Published.

1.4 Revision history

- Version 1.0: 23/12/2025

1.5 Reference Documents

The assignment for this document and all the information included refer to the following documentation:

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the WeBeep page of the Software Engineering II course.
- The Requirement Analysis and Specification Document.

1.6 Document Structure

1. Introduction: it aims to give a brief description of the project. In particular it's focused on the reasons and the goals that are going to be achieved with its development, as previously explained in the RASD.
2. Architectural Design: describes at different levels of granularity the system design implementation.
3. User Interface Design: shows the wire-frames that constitutes the platform UI.
4. Requirements Traceability: explains how the goals defined in the previous document are satisfied by the interaction between the requirements and the component described in this document.
5. Implementation, Integration and Test Plan: provides a description of the implementation integration and testing details, useful for the developers and producer of the platform;
6. Effort Spent: shows the effort spent to write this document.
7. References: contains the references to any documents and software used to write this paper.

2 Architectural Design

2.1 Overview

2.1.1 System View

The BBP system employs a hybrid architectural paradigm, primarily structured around a **3-tier client-server framework**, enhanced by a dedicated **event-driven architecture** layer. This architecture is the perfect balance for addressing the dual requirements of the system: strict transactional user-facing operations and highly responsive asynchronous background processes.

The 3-tier client-server architecture is the fundamental backbone of the BBP system, providing a clear separation between presentation, application logic and data management. This primary structure is used for the synchronous core functionalities made available by the platform, such as account management, bike path browsing, trip planning and history visualization. While the base 3-tier architecture excels at synchronous request-response tasks, some specific functionalities, such as automatic trip tracking, report generation and user notifications, require a more sophisticated asynchronous implementation. These functionalities are handled by the event-driven layer, implemented inside the application layer, which uses asynchronous messaging patterns to decouple the event producing components from their consumers, in order to make the system fast and available to all users.

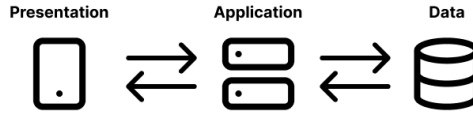


Figure 1: High Level System View. The Event-Driven Layer resides in the Application Layer.

The adoption of the 3-tier client-server architecture offers several key advantages:

- **Modular separation of concerns:** by decoupling the presentation layer from the back-end logic, the system reduces component coupling. This facilitates parallel development streams and simplifies future maintenance.
- **Centralized Security:** authentication, authorization, session-handling and GDPR-related policies are enforced at the application server level, ensuring consistent and controlled access to sensitive data.
- **Improved data consistency:** a centralized data tier simplifies the management of transactional operations and integrity constraints, which is essential for functionalities such as trip history management, deletion of past trips, and performance statistics computation.
- **Scalability and performance control:** the application tier can be scaled independently of the client and data tiers, allowing the system to handle increased loads caused by browsing, filtering, and ranking operations without excessive architectural complexity.

- Controlled integration with external services: interactions with external map and weather services are mediated by the application server, enabling timeout handling, fallback strategies, and graceful degradation in case of service unavailability.

2.1.2 Detailed View

Presentation Tier Users interact with the BBP platform via a mobile application, which constitutes the presentation layer of the system. The client is responsible for rendering the user interface, collecting input data from the user and issuing HTTPS requests to the back-end. The presentation layer does not implement any business rule, as it is only responsible for displaying information collected from the application layer, collecting sensor data and GPS data and triggering actions such as registration, login, browsing, trip planning, report validation and configuration of preferences.

Application Tier The application layer hosts the core services of the BBP system and it's responsible for coordinating all the interactions between each internal service it is composed of, but also between the system and the external services. This layer exposes a set of APIs consumed by the client and it's responsible for enforcing security and privacy constraints, such as authentication, authorization, GDPR-related consent handling, retention rules and secure password storage. From a more practical perspective, the application tier has a micro-service structure, where each component is responsible for a single service:

- Account Management: responsible for registration, login, profile updates, account deletion and user preferences management.
- Bike Path Inventory: responsible for browsing, filtering and getting path data like scores and obstacles.
- Trip Management: responsible for retrieving planned trips, completed trips, but also for the deletion of past trips and the computation of statistics.
- Reporting Management: responsible for manual report creation, automatic report generation, user review, validation and publication.
- Context Enrichment: responsible for the integration of the external weather service and map service, when required.

The application tier also acts as the controlled integration point for the external services, applying timeout handling, input/output validation, and graceful degradation when a provider is unavailable.

The system, as already mentioned, primarily relies on synchronous request-response interactions to support user operations such as authentication, profile management, bike path browsing and trip planning. In this model, each request is handled within a single execution flow, allowing the system to provide immediate feedback to the user. While this approach is well suited for interactive functionalities, it becomes inadequate when applied to operations characterized by continuous data generation, variable computation cost or dependence on external systems. In a purely synchronous implementation, functionalities such as automatic trip tracking, automatic report generation, and trip alert evaluation would require the application server to process data immediately at the moment it is received. This would introduce several issues, including unnecessary resource consumption, increased coupling between system components, and a direct impact on the performance of user-facing services. To overcome these limitations, the application tier integrates an event-driven layer that enables selected functionalities to be handled asynchronously. Instead of forcing immediate processing,

relevant domain occurrences are modelled as events and published to an event bus, where they can be buffered and processed by dedicated consumers independently from synchronous user interactions. In the BBP system, this architectural choice is motivated by the following considerations:

- **Trip Alerts:** in a synchronous design, the system would need to repeatedly check obstacle updates in order to detect relevant changes, resulting in unnecessary resource usage and delayed notifications. By adopting an event-driven approach, alerts are triggered only when reports are published, allowing the system to react only when necessary.
- **Automatic trip tracking ingestion and processing:** tracking data generated by the user’s device may be produced at high and variable frequencies. Processing each data fragment synchronously at the time of generation would directly compete with other user-facing operations, such as browsing or trip planning. Through the event-driven layer, tracking data is published as events and temporarily buffered, enabling the system to decouple data ingestion from processing and to absorb peaks in data production without affecting overall responsiveness.
- **Automatic report generation workflow:** generating an automatic trip report may require aggregating a variable amount of tracking data and performing non-trivial computations. If implemented synchronously at the exact moment a trip ends, this process would slow down the handling of other concurrent requests. By triggering report generation asynchronously, the system can produce the report progressively in background, controlling computational load and ensuring that synchronous services remain responsive.

By decoupling data production from data processing, the event-driven layer improves scalability, fault tolerance through buffering and retries, and system responsiveness, while preserving the overall 3-tier client–server architecture of the BBP system.

Data Tier The data layer is responsible for persistent storage and retrieval of all system data, including user accounts and profile data, consent and tracking preferences, bike paths inventory, planned and completed trips, reports (manual and automatic), and derived statistics. Access to the data tier is mediated exclusively by the application tier, ensuring consistent enforcement of business rules and privacy constraints.

The centralized data tier simplifies consistency requirements, such as ensuring that deleted trips do not appear in history views or aggregated statistics. It also supports reliability through standard mechanisms (e.g., backup, replication strategies, and indexing), while remaining isolated from direct client access.

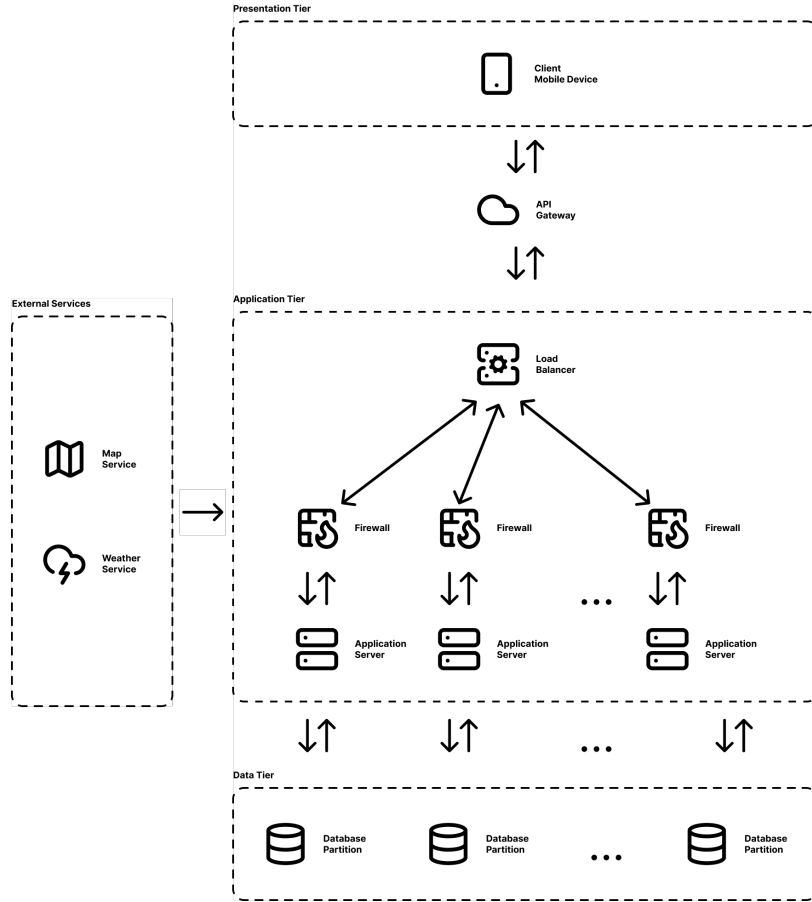


Figure 2: A more detailed view of the BBP system architecture.

2.2 Component View

As already mentioned in the previous section, the BBP system adopts a 3-tier architecture, where the application layer is composed of micro-service components interacting with each other and with external services.

2.2.1 Front-end Components

Mobile Application This component acts as the primary user interface, facilitating all user interactions with the platform via HTTPS requests through the API Gateway. It is responsible for the acquisition and transmission of raw data from device sensors and GPS to the back-end, where the data is processed. Additionally, the application implements dynamic view rendering based on authentication state, restricting non-registered users to path exploration features, while providing registered users with all the available functionalities of the platform.

2.2.2 Application Layer Components

User Management Service This component acts as the centralized authority for identity and access management. It handles the complete lifecycle of the user, such as registration, authentication and credential management, and session control. Also, it allows users to modify their profile information, change their preferences and completely delete their account. All these functionalities are provided to the user while ensuring compliance with the GDPR-related policies and privacy regulations.

Path Exploration Service The Path Exploration Service allows users to browse the inventory of available bike paths, apply filters, search for routes between departure and destination points, and obtain ranked path results based on predefined scores. It returns detailed path information, including status and summaries of reported obstacles.

Trip Management Service This component is responsible for managing the lifecycle of trip data for both planned and completed ones. It implements soft deletion mechanisms to ensure data integrity and view consistency without permanent data loss. Additionally, this service computes performance statistics, such as distance, duration, average speed and other metrics. Crucially, the service enforces strict data isolation, ensuring that trip details and statistics are accessible exclusively to the user who created them.

Report Management Service The reporting component manages both manual and automatic reports as domain entities. In particular it allows publication of reports and validation of automatic report drafts and updates public path information based on the newly published reports, these updates are used to enrich path exploration results and to trigger alert evaluations. Upon publication, the service triggers an update to synchronize and expose the public information of the associated path.

Map & Weather Adapters This component serves as the designated gateway for interactions with external mapping and weather providers. It implements resilience patterns, including timeout handling, fallback strategies, and caching, to ensure system stability. Its primary function is to retrieve weather data to enrich trip information and provides necessary data for map rendering.

Event Bus This component is the asynchronous communication backbone, using a publish subscribe model to dispatch domain events. It decouples event ingestion from downstream processing, ensuring high availability and scalability. This component includes buffering capabilities to absorb load spikes and implements reliability patterns such as automatic retries and back-pressure management.

Tracking Ingestion Endpoint The Tracking Ingestion Endpoint acts as the entry point for tracking data generated by the user's device. It receives GPS points or tracking chunks and immediately publishes corresponding tracking events to the event bus, without performing any processing or persistence, ensuring fast acknowledgements and non-blocking ingestion. No processing or persistence is performed at this stage, the only thing that this component does is to transform those data received by the device in event published in the bus and generate an event (write event name here, the receiver of the event and the purpose)

Tracking Processing Worker The Tracking Processing Worker consumes tracking events from the event bus, produced by the Tracking Ingestion Endpoint, and performs cleaning, normalization, and validation of the collected data. It handles incomplete or inconsistent sensor readings and communicates with the Automatic Report Generator Worker, sharing data for report generation.

Automatic Report Generator Worker The Auto Report Generator Worker reacts to trip completion events. It aggregates processed tracking information and generates automatic report drafts in background, which are later reviewed, validated, and published through the Reporting component.

Alert Service Worker The Alert Service Worker listens for contextual updates such as weather changes and newly published obstacle reports. It evaluates these events against predefined severity thresholds and determines their impact on users' planned trips. When critical conditions are detected, it directly triggers alert notifications for affected users who have requested updates.

For weather-related alerts, the component periodically executes a low-frequency polling mechanism to retrieve meteorological data from external sources through the Map and Weather Integration component. This design choice is motivated by the need to reduce coupling between the alert logic and the weather adapter. Without polling, the weather adapter would be responsible for continuously monitoring weather conditions and actively notifying the Alert Service Worker in case of adverse forecasts, thereby assuming part of the alert evaluation responsibility. By adopting controlled polling, the Alert Service Worker remains the sole component responsible for alert detection logic, while the weather adapter is limited to data retrieval and normalization.

For obstacles alerts, the component listens to the Event Bus for **ReportPublished** events and computes a severity score for a planned trip if there's an obstacle inside the report that interests that same trip. Whenever the computed severity score for a planned trip exceeds a predefined threshold, the Alert Service Worker sends a notification to the corresponding user, ensuring timely and reactive alert delivery without introducing unnecessary coupling or continuous monitoring overhead in other components.

2.2.3 Data Layer

Database Management System This component acts as a mediator between the components of the application layer and the databases. It is responsible for managing all domain data, including user profile data and preferences, path information, planned and completed trips, tracking data, reports and any other aggregated data required for proper functioning of the system.

2.2.4 External Services

Map Service This component is a third-party geographical data provider used by the platform. It supplies the assets for map visualization (vector or raster tiles) and other geographical features.

Weather Service This component is a third-party meteorological data provider used by the platform. It supplies access to the current atmospheric conditions, and severe weather warnings for specific geospatial coordinates. This data is consumed via API to enrich trip details and trigger safety alerts for users.

User Device This component is the physical hardware interface and operating system services of the user's mobile device. It serves as the primary source of telemetry, continuously generating geolocation coordinates (via GPS) and motion sensor data. Crucially, it enforces operating system-level security, ensuring that the application can only access this sensitive hardware if the user has explicitly granted the necessary runtime permissions.

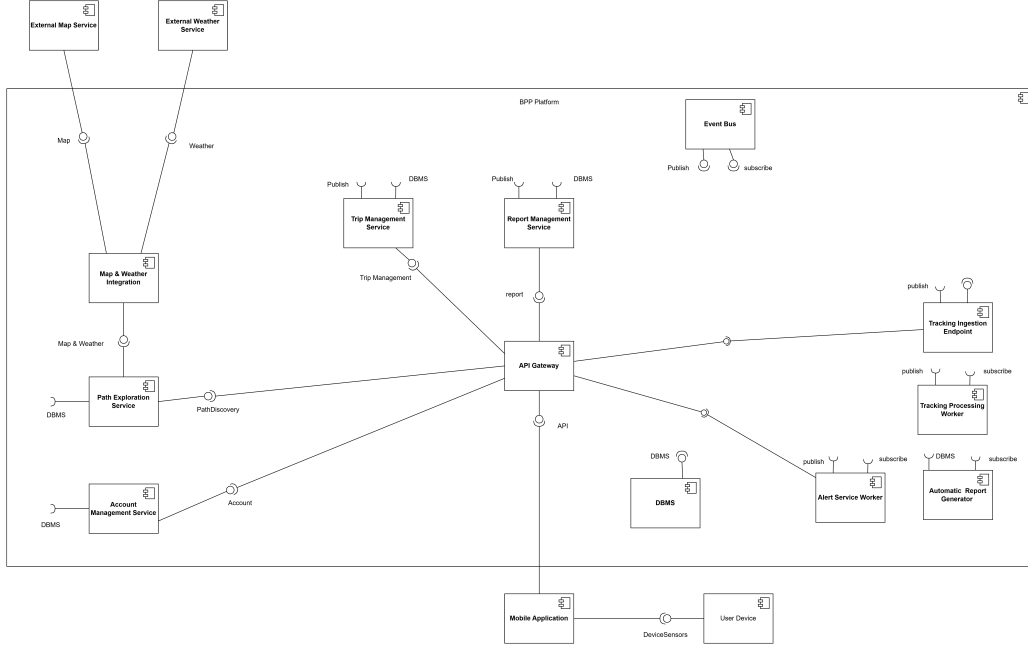


Figure 3: Component Diagram.

2.3 Deployment View

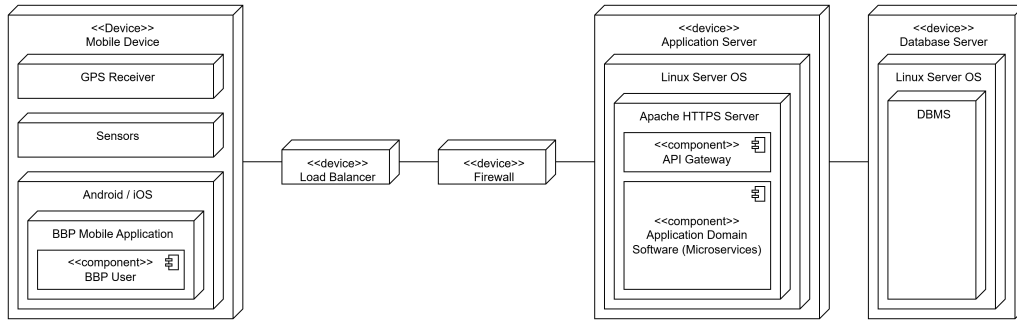


Figure 4: System Deployment View.

Mobile Device The device is used by the BBP users to access the platform, via the mobile application. The user is required to download the application from centralized distribution channels

like Google Play Store (for Android) or App Store (for iOS), in order to interact with the system. Also, the device is responsible for gathering all the GPS and sensor data, required for the correct functioning of specific features of the application.

All data exchange between the mobile client and the server infrastructure is encapsulated over HTTPS, passing through a load balancer, for high availability, and firewalls, for robust security. Requests are finalized at the API Gateway, which acts as the single entry point for the back-end, handling request routing, rate limiting, and authentication before passing data to the internal application servers.

Load Balancer The load balancer acts as a reverse proxy, distributing incoming network traffic across the servers that constitute the application layer. Its primary purpose is to ensure high availability and reliability by preventing that a single server may become a point of failure or a performance bottleneck, slowing down the entire system.

Firewall The firewall is a network security system that acts as a protective barrier between the trusted internal network (BBP platform back-end) and the untrusted external network. Its primary purpose is to monitor and control incoming and outgoing traffic based on predetermined security rules to prevent unauthorized access and block malicious data. Notoriously, the deep packet inspection required by the firewall represents a potential performance bottleneck. Therefore, the firewall is strategically positioned after the load balancer to reduce the volume of packets it has to inspect.

Application Server The application server handles all the incoming requests from users interacting with the platform via the mobile application. It is responsible for processing data sent by the client and ensures data persistence by interacting with the DBMS via TCP/IP. It also integrates information coming from external services with which it communicates via HTTPS.

Database Management System Server The database runs on a clustered architecture to ensure high availability and data consistency. This configuration guarantees that all nodes maintain identical dataset and that the application server is updated with the latest information, preventing errors caused by data mismatches between nodes.

2.4 Runtime View

The following diagrams show the interaction between the various component that make up the BBP system. Note that the message flow modelled by these sequence diagrams do not include all the possible exceptions the system can run into for a matter of simplicity. Furthermore, as already mentioned before, every interaction between the client and the server is mediated by the API Gateway component, which also has caching mechanisms and acts as a session handler for registered users. These functionalities are not shown in the diagrams to make them more readable.

Before continuing with the analysis of the sequence diagrams, we introduce the following events used by asynchronous components:

1. **TrackingDataReceived (TDR)**: produced by Tracking Ingestion Endpoint component that transform the data sensed by the GPS of the device into events that will be stored in the bus waiting to be processed.
2. **TrackingDataProcessed (TDP)**: produced by the Tracking Processing Worker after cleaning the data to notify the Automatic Report Generator Worker that the data is ready to be further processed for report compilation.

3. **TripCompleted (TC)**: produced by the Trip Management Service to report that the user has completed a trip. This event is then managed by the Automatic Report Generator that produces a draft of the report, persisted in the database.
4. **ReportPublished (RP)**: produced by the Report Management component upon report publication. It triggers the Alert Management component, which recalculates the severity scores of the paths in the report and notifies users if needed.

UC1: User Registration

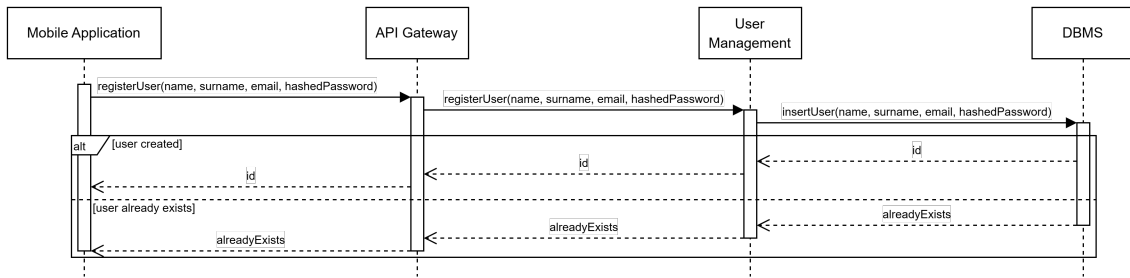


Figure 5: User Registration Sequence Diagram.

This sequence diagram shows the interactions between system components during user registration. Note that the local email and password validation is only syntactical, checking for errors in the email formatting and for well formed passwords. The password is hashed using salting and peppering, as already mentioned in the RASD, before sending the registration request to the back-end for better security. The `id` is generated by the DBMS, cached in the API gateway and returned to the user for session handling: all subsequent client requests also include the `id`, so the API gateway can authenticate the user.

UC2: User Login

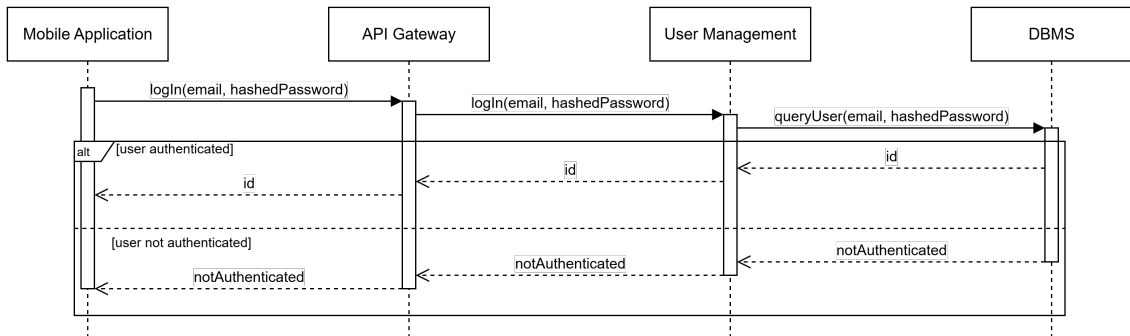


Figure 6: User Login Sequence Diagram.

This sequence diagram shows the interactions between system components during user login. If

the email and hashed password correspond to an entry of the database, the corresponding id is cached in the API gateway and returned to the client, so all subsequent requests can be authenticated.

UC3: Update Profile Data

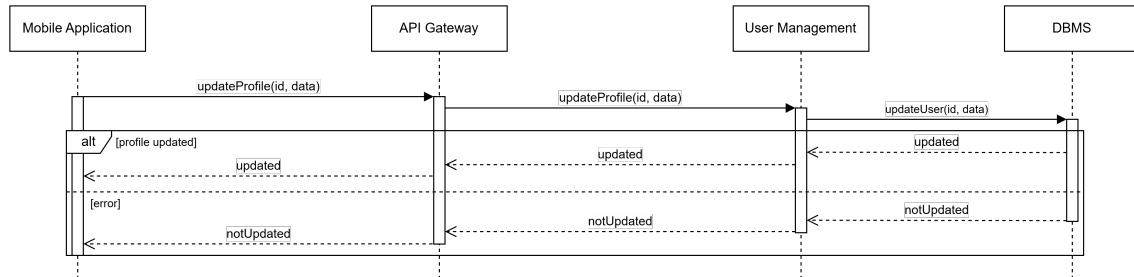


Figure 7: Update Profile Data Sequence Diagram.

This sequence diagram shows the interactions between system components during profile data updates. The client sends an update request to the server, which then responds with a success or failure message.

UC4: Delete Account

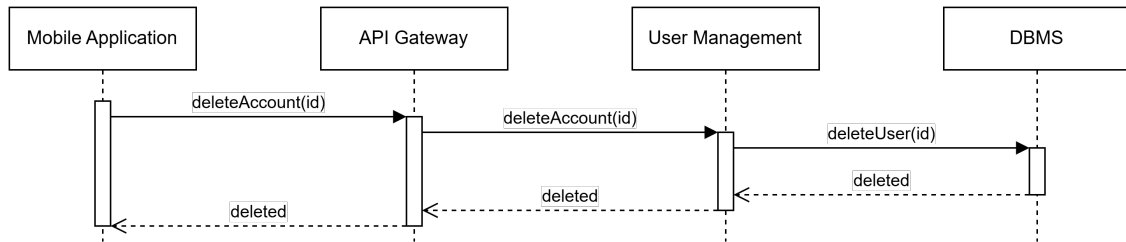


Figure 8: Delete Account Sequence Diagram.

This sequence diagram shows the interactions between system components during account deletion. By GDPR regulations, all sensitive information regarding the user is deleted, while all reports he compiled are persisted in the database.

UC5: Browse Bike Paths

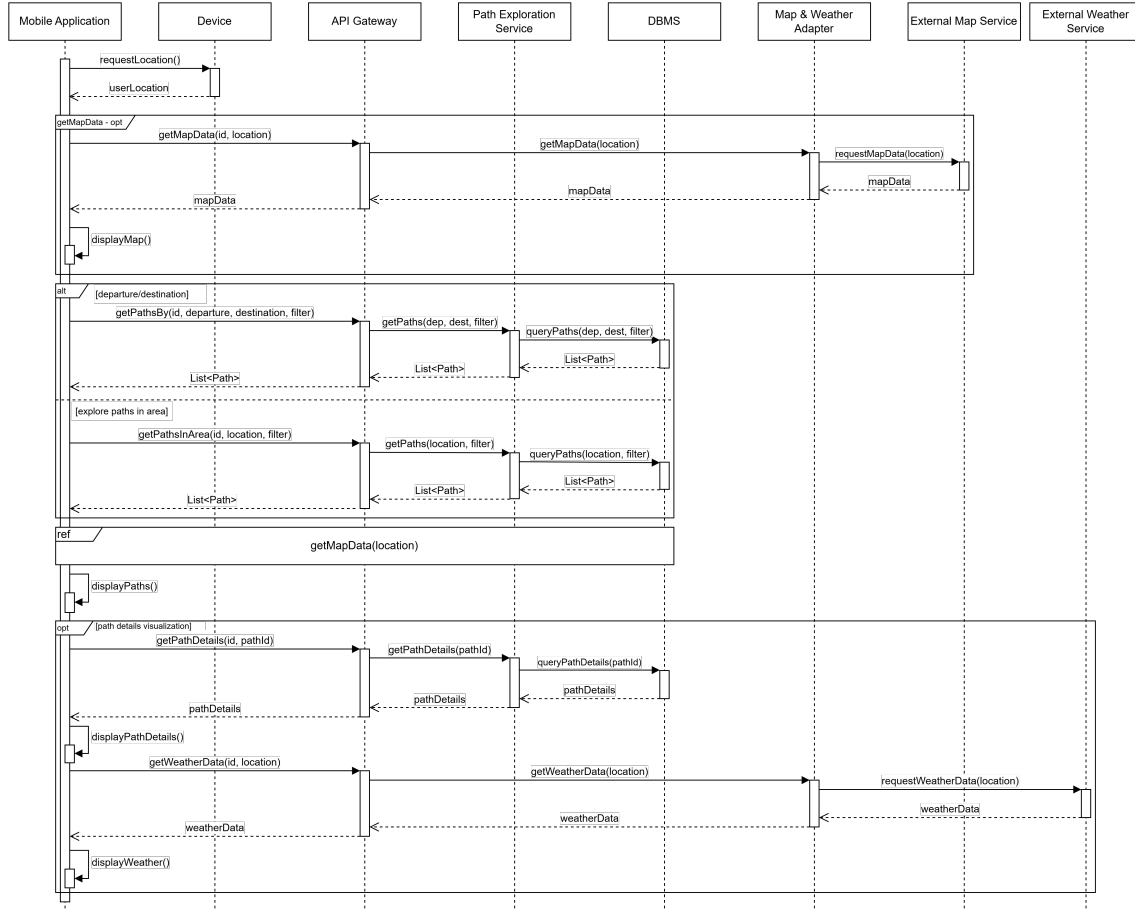


Figure 9: Browse Bike Paths Sequence Diagram.

This sequence diagram shows the interactions between system components during bike paths browsing. Once the user navigates to the dedicated section in the mobile application, a location request to the device is dispatched. When the user location is returned by the device, the application gets map information from the external service, in order to display the map on the screen. If the user inserts departure and destination points, with attached filters, a request to the server is dispatched, which returns the list of the available bike paths, ranked by score. Otherwise, if the user explores the map, a request by area is sent to the server. In both cases, the map information is updated and the bike paths are displayed.

The user can optionally visualize the details of a given path by clicking on it, which not only retrieves all the path information, but also the weather information on the area regarding the bike path.

UC6: Plan Trips

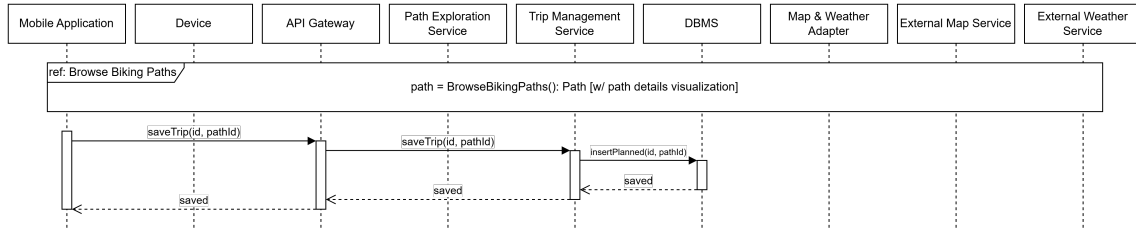


Figure 10: Plan Trips Sequence Diagram.

This sequence diagram shows the interactions between system components during trip planning. After browsing and selecting a bike path, the user can decide to save the trip: the client sends a request to the trip management service, which saves the trip in the database via the DBMS.

UC7: Change Automatic Tracking Settings

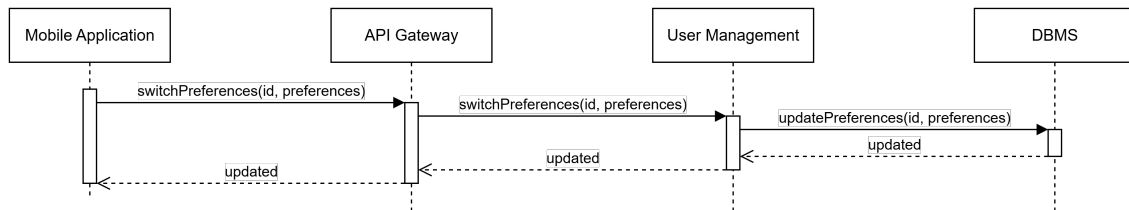


Figure 11: Change Automatic Tracking Settings Sequence Diagram.

This sequence diagram shows the interactions between system components during preferences changes. The client simply dispatches a request to switch preferences to the user management component, which saves the preferences to the database, via the DBMS.

UC8: View History and Statistics of Completed Trips

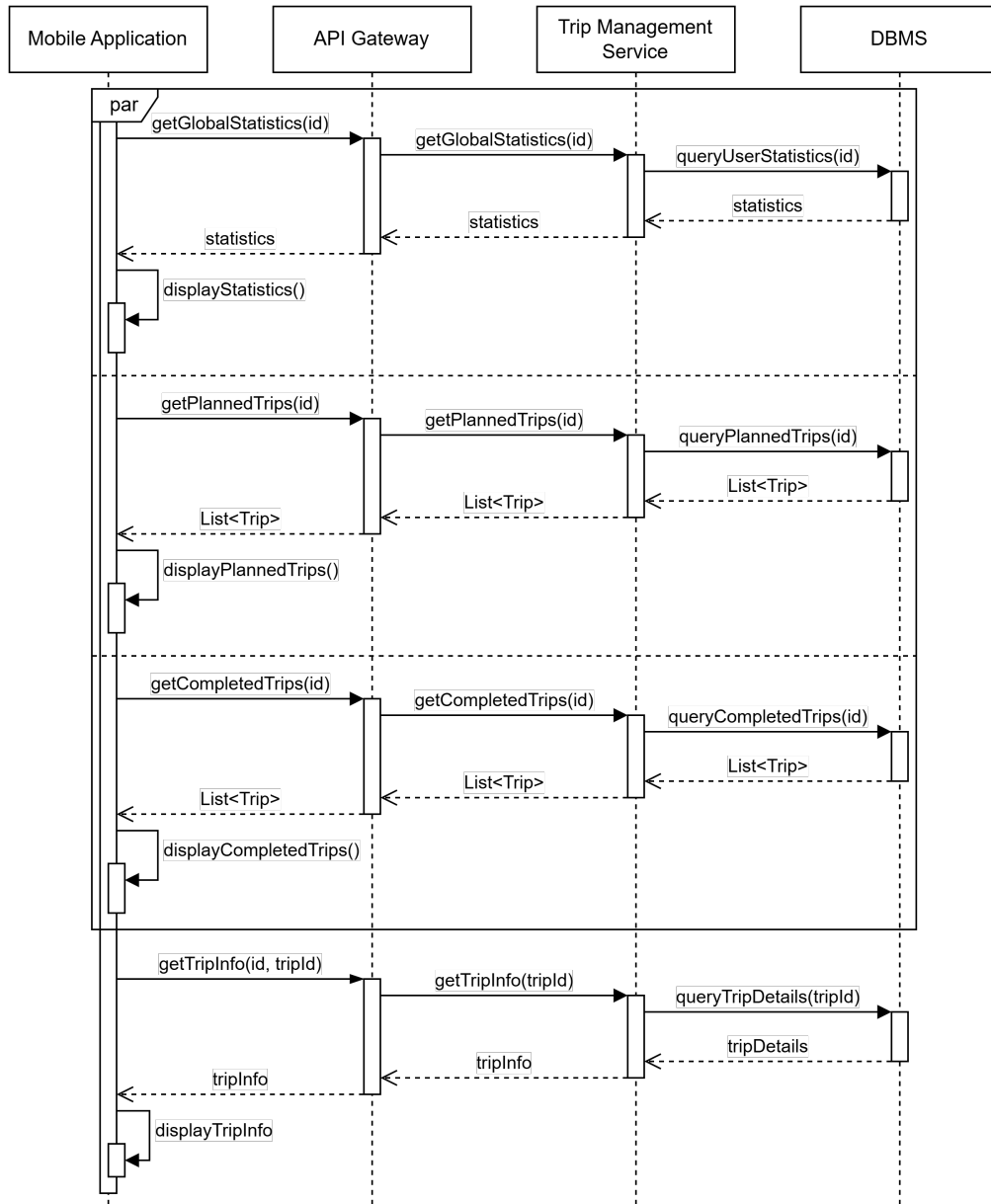


Figure 12: View History and Statistics of Completed Trips Sequence Diagram.

This sequence diagram shows the interactions between system components when the user views the history and statistics of completed trips. The client gets from the server the global user statistics, all planned and completed trips in parallel, showing them to the user. Once the user clicks on a completed trip, all information regarding the trip is fetched and displayed.

UC9: Delete Trips from History

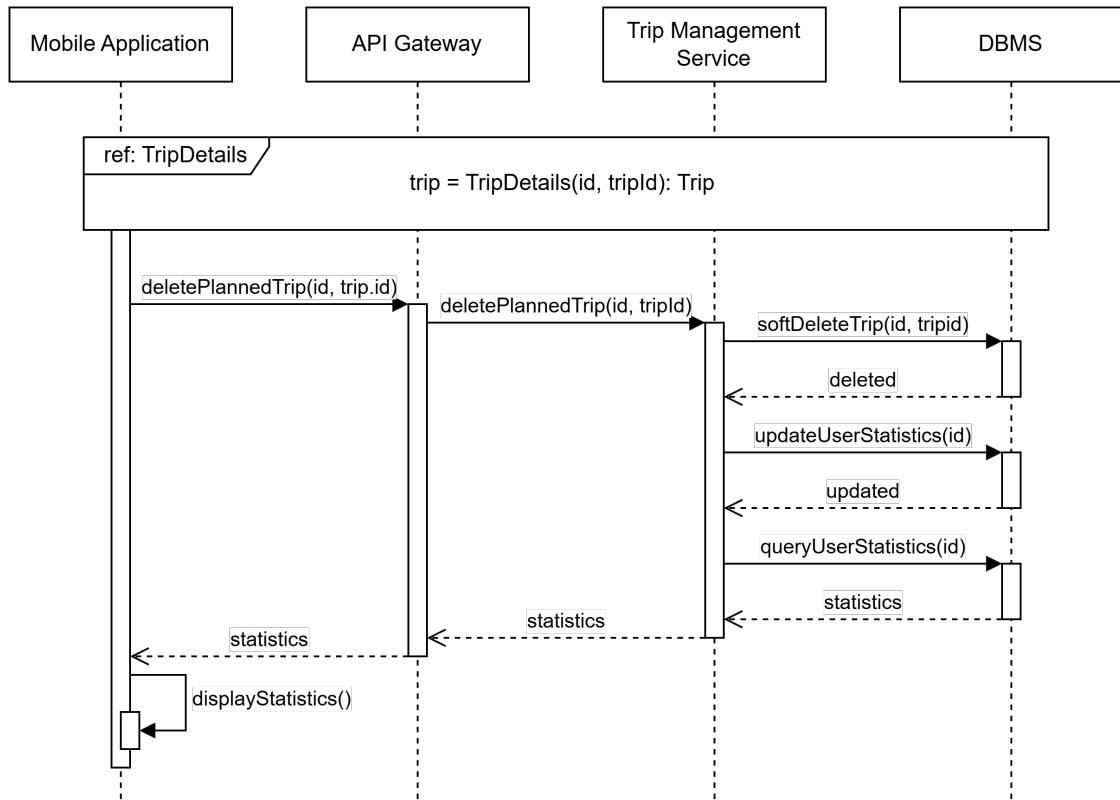


Figure 13: Delete Trips from History Sequence Diagram.

This sequence diagram shows the interaction between system components during trip deletion. The user can decide to delete a trip after visualizing its details. The client sends a request to the trip management service, which interacts with the DBMS by requesting a soft deletion of the information, the update of user global statistics and the query of the new ones, so the user can visualize them.

UC10: Manual Report

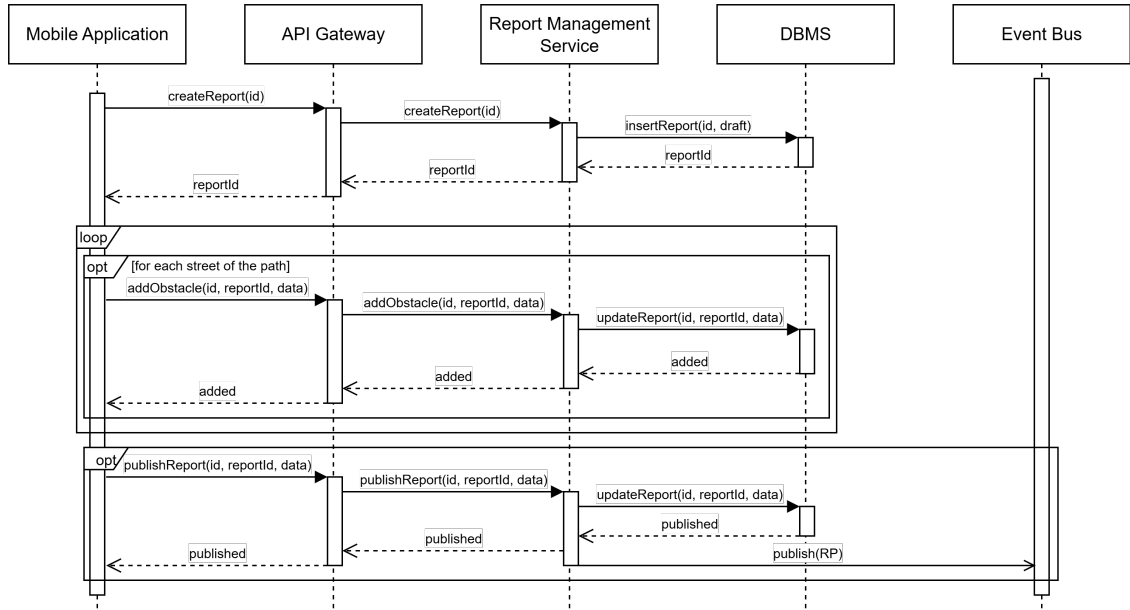


Figure 14: Manual Report Sequence Diagram.

This sequence diagram shows the interaction between system components during manual reporting. After completing his trip, the user can compile the report regarding the bike path he followed. A creation request is dispatched from the client to the report management system, which creates a new draft and inserts it into the database via the DBMS. For each street of the path, the user can add a new obstacle, which is done once again via a request to the report management service. Once the validation process is completed, the user may optionally publish the data: the client sends a request to the report management component which updates the state of the report, and publishes a new **ReportPublished** (RP) event on the bus, which triggers a synchronization process in the application layer to make the data up-to-date.

UC11: Automatic Report

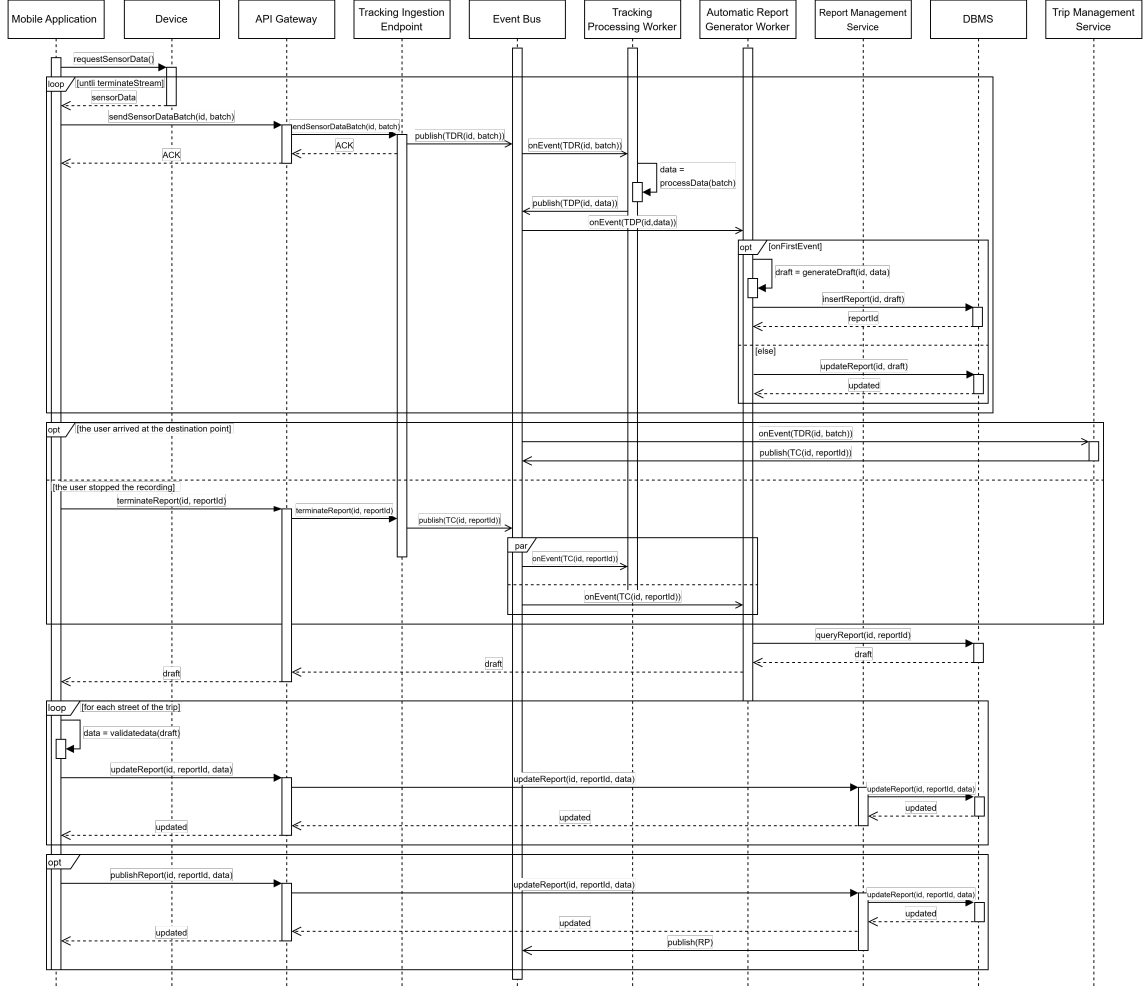


Figure 15: Automatic Report Sequence Diagram.

This sequence diagram shows the interactions between system components during automatic report generation. Once the user starts his trip, the mobile application receives the sensor data, including GPS location, from the device. The client batches all the data and sends it to the tracking ingestion endpoint, which simply publishes to the bus a **TrackingDataReceived** (TDR) event containing the batch. The tracking processing worker reacts to this event by cleaning the data and publishing in turn a new **TrackingDataProcessed** (TDP) event to the bus. This triggers a sequence of tasks in the automatic report generator worker, which creates a new report draft, inserts it into the database, via the DBMS component, and updates it when a new TDP event is captured from the bus. When the user reaches the destination point, or stops the recording of the trip, a termination event is published into the bus by either the trip management component or the tracking ingestion endpoint respectively. This event is consumed by the automatic report generator, which queries the

draft from the DBMS and sends it to the client, so the user can validate the report. Every validation step executed by the user triggers an update to the database, until the verification is completed. At this point the user may optionally publish the data: the client sends a request to the report management component which updates the state of the report, and publishes a new event on the bus, which triggers a synchronization process in the application layer to make the data up-to-date.

UC12: Receive Trip Alerts

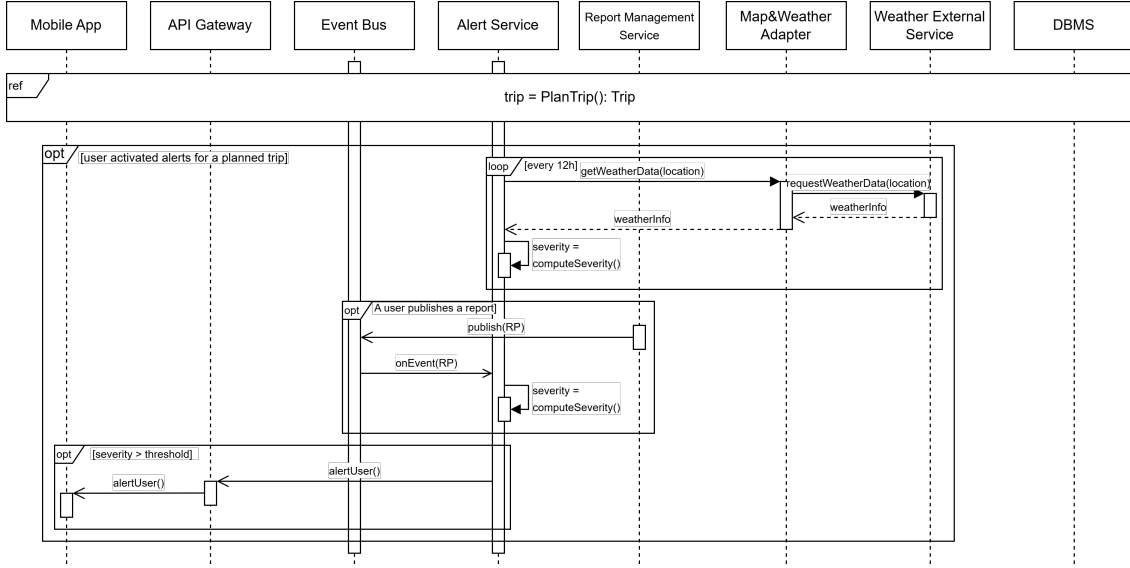


Figure 16: Receive Trip Alerts Sequence Diagram.

This sequence diagram shows the interactions between system components during trip alerts. Once the user has planned a trip, the system activates a polling mechanism that every 12 hours requests weather information to the external weather provider, via the adapter. The alert component, upon receiving the weather data, computes a severity score. Also, every time the report management service publishes a **ReportPublished** (RP) event on the bus, the alert service reacts by recomputing the severity score. When the score exceeds a given threshold, the alert service triggers a notification to the user's device.

2.5 Component Interfaces

All internal system components interact with each other via gRPC, which is a highly efficient protocol to minimize communication latency. Note that in all previous diagrams the `id` passed in almost every function call is the `id` of the user requesting the represented operations. Technically, the communication between the client and the server is done via the HTTPS protocol, which includes the `id` in the body, while in internal procedure calls it is necessary to pass it as argument to perform the requested operations.

API

- register(name, surname, email, hashedPassword)
- login(email, hashedPassword)
- deleteAccount(id)
- getMapData(id, location)
- getWeatherData(id, location)
- getPathsBy(id, departure, destination, filter)
- getPathsInArea(id, location, filter)
- getPathDetails(id, pathId)
- saveTrip(id, pathId)
- switchPreferences(id, preferences)
- getGlobalStatistics(id)
- getPlannedTrips(id)
- getCompletedTrips(id)
- getTripInfo(id, tripId)
- deletePlannedTrip(id, tripId)
- createReport(id)
- addObstacle(id, reportId, data)
- publishReport(id, reportId)
- sendSensorDataBatch(id, data)
- terminateAutomaticReport(id, reportId)
- updateAutomaticReport(id, reportId, data)

User Management Service Interface

- registerUser(name, surname, email, hashedPassword)
- logIn(email, hashedPassword)
- updateProfile(id, data)
- deleteAccount(id)

Path Exploration Service Interface

- `getPaths(departure, destination, filter)`
- `getPaths(location, filter)`
- `getPathDetails(pathId)`

Trip Management Service Interface

- `saveTrip(id, pathId)`
- `getGlobalStatistics(id)`
- `getPlannedTrips(id)`
- `getCompletedTrips(id)`
- `getTripInfo(tripId)`
- `deletePlannedtrip(id, tripId)`

Report Management Service Interface

- `createReport(id)`
- `addObstacle(id, reportId, data)`
- `deleteObstacle(id, reportId, data)`
- `publishReport(id, reportId, data)`
- `updateReport(id, reportId, data)`

Map & Weather Adapter Interface

- `getMapData(location)`
- `getWeatherData(location)`

Event Bus Interface

- `subscribe()`
- `publish(Event)`

Tracking Ingestion Endpoint Interface

- `sendSensorDataBatch(id, data)`

Tracking Processing Worker Interface The tracking processing worker component only reacts to `Sense(id, batch)` events on the bus, so it does not have an interface, other than the one needed for the publish-subscribe pattern.

Automatic Report Generator Worker Interface The automatic report generator worker component only reacts to `PSenseE(id, data)` events on the bus, so it does not have an interface, other than the one needed for the publish-subscribe pattern.

Alert Service Worker Interface

- `alertUser()`

Database Management System Interface

- `insertUser(name, surname, email, hashedPassword)`
- `queryUser(email, hashedPassword)`
- `updateUser(id, data)`
- `deleteUser(id)`
- `queryPaths(departure, destination, filter)`
- `querypaths(location, filter)`
- `queryPathDetails(pathId)`
- `insertPlannedTrip(id, pathId)`
- `updatePreferences(id, preferences)`
- `queryUserStatistics(id)`
- `queryPlannedTrips(id)`
- `queryCompletedTrips(id)`
- `queryTripDetails(tripId)`
- `softDeleteTrip(id, tripId)`
- `updateUserStatistics(id)`
- `insertReport(id, draft)`
- `updateReport(id, reportId, data)`
- `queryReport(id, reportId)`

2.6 Selected Architectural Styles and Patterns

3 Tier Architecture The BBP system is designed upon a robust three-tier client-server architecture, enhanced by an event driven layer. This design decouples the interactions the user has with the platform from the services it provides, without slowing down the user experience, as heavy computational tasks can be processed in the background avoiding latency. The resulting loose coupling brings great flexibility and facilitates horizontal scalability of the back-end as components can be deployed as micro-services, which can be replicated across distributed servers to accommodate for the growing user demand. Also, it guarantees grater security, as every component of the application layer is developed to comply with the GDPR regulations and best security practices.

Event-Driven Architecture Most of the interactions between the client and the server, but also between the components of the application layer, are synchronous, but there are some critical features that require a more complex architecture: hence the need to add an asynchronous event driven layer, which allows some heavy computational tasks to be processed in the background without interrupting the user experience flow. More specifically, this hybrid architecture features an event bus component which streams a queue of messages other components of the system can listen and react to. This allows low latency components to communicate with heavy duty ones, without the need of complex synchronization between them.

Microservice Architecture As already mentioned, the microservice architecture design best fits the need of a resilient and scalable system to allow the user to interact with the feature-rich BBP platform. The presentation layer communicates with the components the application layer is made of via the API gateway, which constitutes the entry point of the back-end service, allowing the client an easy and controlled access to the platform. Microservices communicate with each other via a high efficiency protocol, like gRPC, allowing faster data retrieval and a more structured and robust system, without compromising flexibility. Also, the microservice architecture allows the deployment of the components across a distributed system, so it is easier to horizontally scale up the back-end to accommodate for growing user demand.

Circuit Breakers The system requires to be highly available to the end users of the platform, to ensure uninterrupted service at all times. To achieve this objective, the circuit breaker pattern is integrated within the application layer to prevent cascading failures, effectively isolating faults when service latency or failures are detected.

State Pattern The central core of the system is obstacle information, which is gathered by means of reports compiled by users. Careful management of this data is therefore crucial to ensure the system functions correctly. To achieve this objective, the management of the reports is implemented using the state pattern, which also minimizes data loss.

Publish Subscribe Pattern As already mentioned in previous sections, the asynchronous nature of some features of the platform led to the adoption of an hybrid architecture with an Event-Driven layer, implemented using a publish-subscribe pattern. More specifically, asynchronous components subscribe to the event bus, listening to the stream of events and reacting to those that concern them. This design decision prevents tight coupling of components and makes asynchronous tasks effortless.

Adapter Some features of the platform rely on external services, such as map and weather solutions, to enrich the user experience. The adapter pattern becomes a necessity as data from external services have a different structure from that used by the system. Some components of the application layer, namely the Map and Weather Adapter, act as a link between the external and the internal services, providing an interface through which the platform is able to communicate. It also allows timeout handling, fallback strategies, and graceful degradation in case of service unavailability.

3 User Interface Design

The following figures show a mock-up user interface of the mobile application for the Best Bike Paths system. The images exhibit the most important screens from the perspective of a registered user. The home screen shown in figure 18 and the profile screen shown in figure 21, are replaced by a direct link to the registration page if the user did access the application without signing up.

Sign In and Sign Up Pages In this screen the user can sign in by entering e-mail and password, if already registered. If the user is new to the platform, can click the *"New to the community?"* button and access the sign up screen, where he can enter his name, surname, email and password to create an account. If a registered user accidentally entered the sign up screen, can go back to the sign in screen by clicking the *"Already part of the community?"* button. In both screens, non-registered users can access the map screen shown in figure 19 by clicking the *"Let's explore without account"* button.

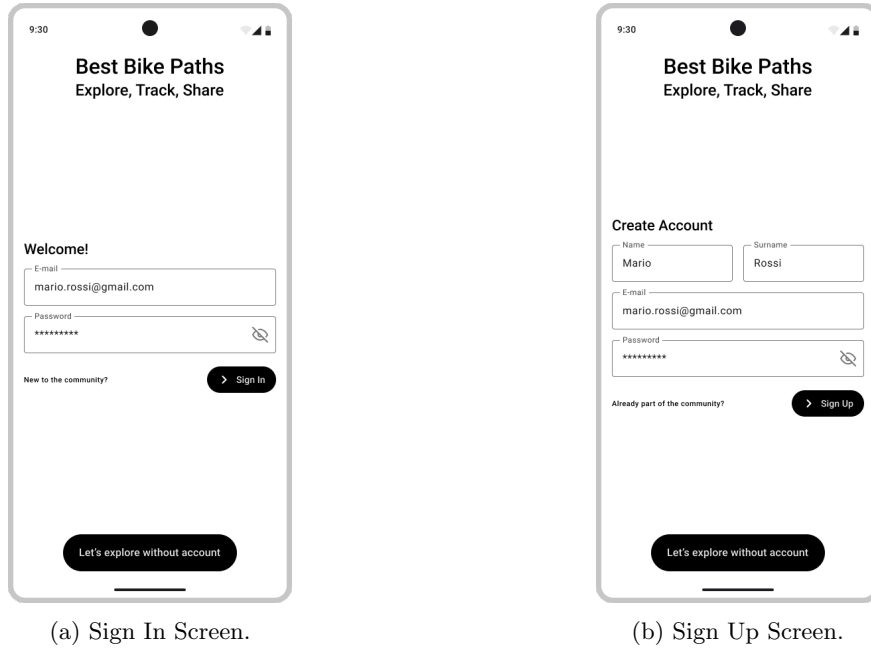


Figure 17: Sign in and Sign up screens.

Home Screen In this screen the registered user can visualize his statistics, the trips he planned and all his biking activities. By clicking on a saved trip, the user can immediately start the navigation of the selected trip, as shown in figure 19d. By clicking on an activity, the user can see the details regarding that specific activity, like total distance, average speed, total time, elevation and weather conditions, and also the report of all obstacles encountered during his trip. In this screen the user can also delete the activity by clicking on the trash button (top right corner), after confirmation, or go back to the home page by clicking the back button (top left corner). This screen is reserved to registered users only.

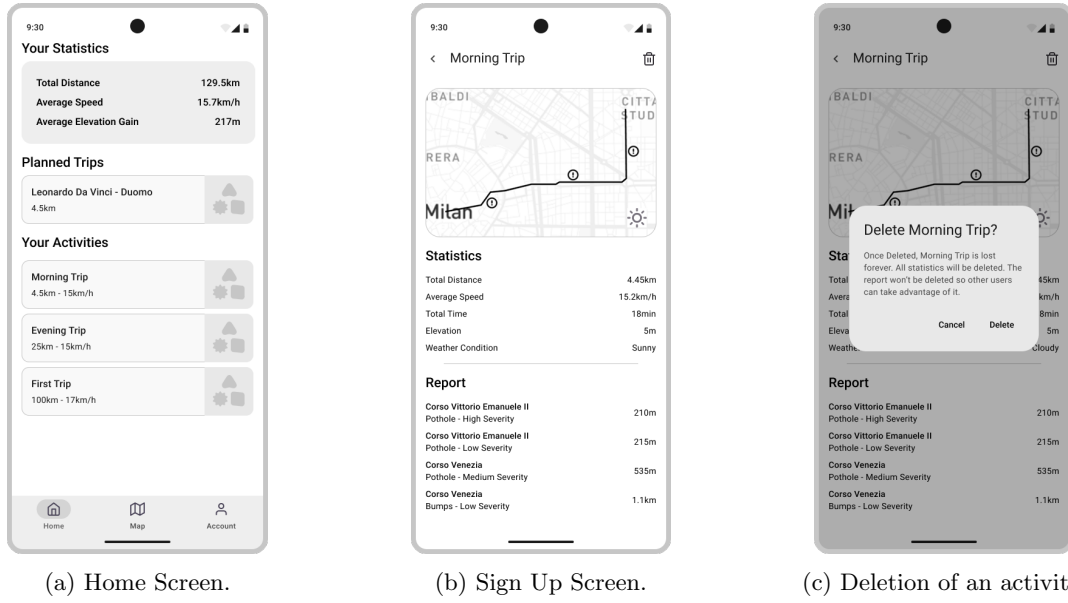


Figure 18: Home Screen and Trip Details Screen.

Map Screen In this screen users can browse the map, visualize their current position on the map, by clicking the first button of the floating menu on the bottom left corner, and change the map orientation by clicking to the second button of the floating menu. By inserting a departure and destination point in the first and second input fields respectively, the user can see on the map different paths that lead from the departure point to the destination one. By clicking on a path, the user can see the major obstacles along the way signalled by other users and the score of every segment of the path coded in different colours.

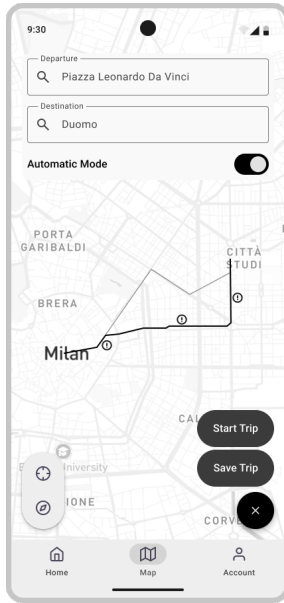
The following is reserved to registered users only. When clicking on a path, a switch appears under the input fields, allowing the user to activate automatic data acquisition when cycling. Also a floating action button on the bottom right corner appears, so the user can start biking the selected route. Once clicked, the floating action button reveals two options: "Save Trip" or "Start Trip", which let the user choose between saving the trip for later cycling sessions, or immediately starting the trip. By pressing the "Save Trip" button, the user can visualize his trip in the home screen, shown in figure 18a. By pressing the "Start Trip", the user is taken to the navigation page, where he can stop the recording at any time by clicking on the "Stop Recording" button, add a new obstacle on his way by clicking the bottom right floating action button with the plus sign. or switch between manual or automatic mode, by clicking the switch in the bottom left corner.



(a) Map Screen.



(b) Map Screen Trip Planning.



(c) Map Screen Trip Selection.



(d) Map Navigation.

Figure 19: Map Screens and Navigation.

Report Screen This screen is presented to registered users who terminated their biking session. Here can manually add obstacles found along the way, or review the ones added with the floating action button with the plus sign in the bottom right corner found in the previous screen, shown in figure 19d. Here can manually modify the name of the obstacle, the street where the obstacle

was found and the severity of the obstacle. Also, the registered user is suggested with obstacles signalled by other members of the community, so it's easier for him to insert obstacles. The user can also delete obstacles he accidentally inserted, by clicking in the trash icon button next to the name of the obstacle. The same set of operations can be performed by a registered user who chose to automatically gather information when presented with the floating action button option in screen 19c. This screen is presented to the user for every street he went through during his journey. He can go the next route by pressing the "Next" button or go to the previous one by clicking on the "Previous" button. Once the review of the report is completed, the user is presented with a greeting screen, where he can visualize his statistics and go to the home screen. This screen is reserved to registered users only.

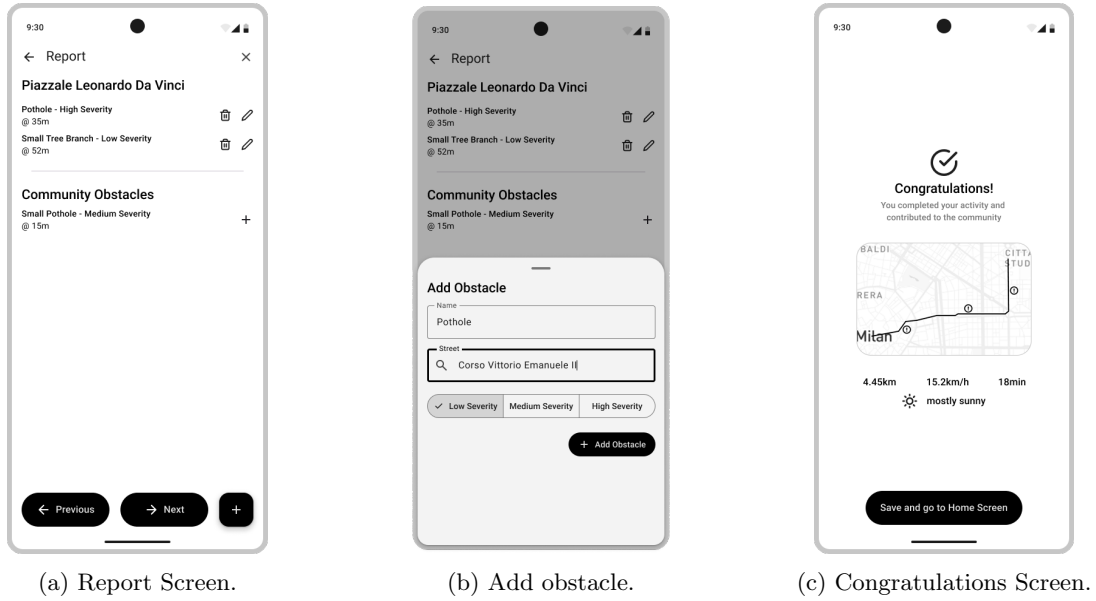


Figure 20: Report and Congratulation Screens.

Profile Screen In this screen the user can see his personal information and the number of activities and contributions he has made. Also, he can change his preferences via the toggle button or delete his account via the trash button. This screen is reserved to registered users only.

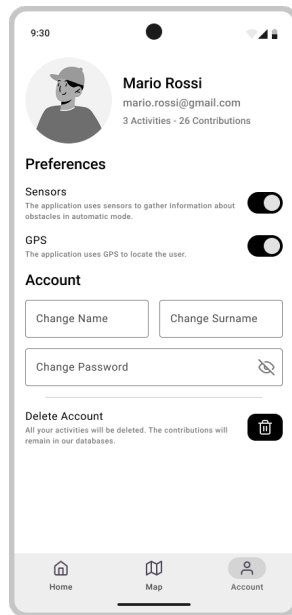


Figure 21: Profile Screen.

4 Requirements Traceability

Requirements	• R1: The system shall allow biker to create an account. • R2: The system shall allow registered users to log in. • R3: The system shall allow registered users to update information in their profile. • R4: The system shall allow registered users to delete their account.
Components	• Mobile Client • API Gateway • User Management Service

Table 1: Account and Access Management Mapping.

Requirements	• R5: The system shall allow users to browse the inventory of paths • R6: The system shall allow users to visualise available bike paths on a map provided by an external map service.
Components	• Mobile Client • API Gateway • Path Exploration Service • Map & Weather Integration • DBMS • External Map Service

Table 2: Bike Path Discovery and Visualisation Mapping.

Requirements	• R7: The system shall allow registered users to plan a bike trip by storing it among planned trips, • R8: The system shall allow the user to specify a starting and ending point in order to retrieve all the possible path between them. • R9: In case a registered user has provided a departure and a destination point, the system shall return the possible paths ordered based on their scores.
Components	• Mobile Client • API Gateway • Path Exploration Service • Map & Weather Integration • Trip Management Service • DBMS • External Map Service

Table 3: Trip Planning and Route Recommendation Mapping.

Requirements	• R10: The system shall allow registered users to record their trips. • R11: The system shall provide registered users with statistics about their cycling performance. • R12: The system shall allow registered users to view all the non deleted past trips. • R13: The system shall allow registered users to delete trips from their trip history. • R14: The system shall allow registered users to enable or disable automatic trip tracking.
Components	• Mobile Client • API Gateway • Trip Management Service • Event Bus • Tracking Ingestion Endpoint • Tracking Processing Worker • DBMS • User Device (Sensors/GPS) • User Management Service

Table 4: Trip Tracking, History and Performance Analytics Mapping.

Requirements	• R15: The system shall allow registered users to manually produce a report. • R16: The system shall allow registered users to automatically produce a report. • R17: The system shall allow registered users to review and validate reports produced automatically. • R18: The system shall allow registered users to publish manual reports. • R19: The system shall allow registered users to publish validated reports produced automatically.
Components	• Mobile Client • API Gateway • Reporting • Event Bus • Auto Report Generator Worker • DBMS

Table 5: Reporting, Validation and Publication of Path Information Mapping.

Requirements	• R20: The system shall notify registered users with planned trips when relevant weather conditions affecting the trip exceed a predefined severity threshold. • R21: The system shall notify registered users with planned trips when obstacles along the associated path exceed a predefined severity threshold. • R22: The system shall enrich trips in the inventory with meteorological information retrieved from an external weather service when available.
Components	• API Gateway • Map & Weather Integration • Event Bus • Alert Evaluation Worker • DBMS • External Weather Service

Table 6: Context Enrichment and Notifications Mapping.

5 Implementation, Integration & Test Plan

5.1 Implementation Plan

The system will be implemented using a thread strategy, which ensures that a core platform feature is fully functional before moving to the next one, delivering at each step of the implementation an intermediate product stakeholders can evaluate and validate. This strategy requires the identification of the system threads to be incrementally implemented and tested, based on the requirements outlined in the RASD.

T1: User Management This thread allows users to create a new account via email and password, login to an existing one using their credentials, update their personal information, update their preferences, for instance sensor or GPS acquisition, and delete their account.

T2: Path Exploration This thread allows the user to browse all the biking paths in a certain area, or only the ones connecting a departure and destination point, ranked by their status, and filtered based on their requirements, if any. The users can also visualize all the details regarding a given bike path, like obstacles along the way.

T3: Trip Management This thread allows the user to visualize their global statistics, including total travelled distance, average speed and more, save bike paths as planned trips and visualize all their completed trips, with the relative statistics.

T4: Reporting This thread allows the user to manage the lifecycle of manual and automatic reports obstacles. More specifically, allows the user to change the status of manual reports from **draft** to **published** and the status of automatic reports from **draft** to **validated** and then **published**.

T5: Alert Management This thread allows the user to change their notification preferences and receive notifications about adverse weather or road conditions regarding their planned trips.

T6: Integration Adapter This thread has as its only objective to test the communication between the platform and the external map and weather services.

T7: Tracking Pipeline This thread has as its only objective to test the interaction with the device of the user and the management of sensor and GPS data.

5.2 Component Integration and Testing

This testing plan follows an incremental strategy, where features are introduced and validated in stages. Each phase involves testing new components or extending the evaluation of previously integrated modules. We assume that the DBMS is fully functional at each step, so it's not depicted in this plan. Furthermore, external services are included in the diagram as we can test the communication between them and the adapters.

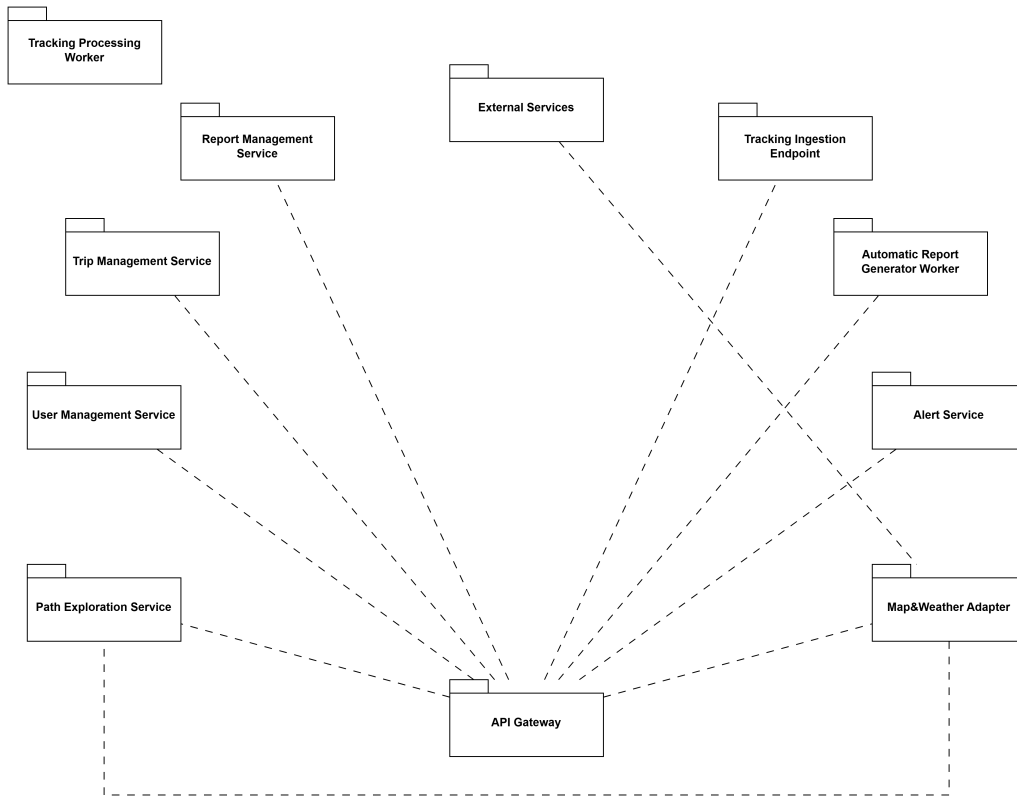


Figure 22: Test Plan Diagram.

First Incremental Test This increment focuses on validating the core user onboarding functionalities, which are a prerequisite for accessing all other system features. The tested functionalities include user registration, authentication, profile data updates, and the management of consent and tracking preferences. From an architectural perspective, this increment primarily involves:

- the client, which collects user input and initiates the interactions
- the API Gateway, which acts as the entry point and routes requests
- the User Management Component, which handles identity management, credential validation, profile persistence, and consent storage in compliance with GDPR requirements.

At the end of this increment, the system provides a fully functional and stable user management flow that can be used as a foundation for subsequent increments.

Second Incremental Test This increment validates the integration of external map services used to support path visualization within the system. The tested functionalities include the rendering of map views associated with bike paths and the correct handling of failure scenarios, such as timeouts or unavailability of the external map service, through fallback mechanisms.

The components involved in this increment are

- the client, responsible for displaying the map to the user

- the API Gateway, which routes the requests
- the Path Exploration Service, which provides the path data to be visualized
- the Map & Weather Integration component, which encapsulates communication with the external map service and enforces resilience policies.

This increment ensures that map-related features are robust and correctly isolated from external service failures before extending the system with additional context-aware functionalities.

Third Incremental Test This increment focuses on validating the core path exploration functionalities that do not depend on external services.

From an architectural standpoint, the client initiates the exploration requests, optionally involving the User Management Component when the user is authenticated, although these functionalities are also accessible to non-registered users. Requests are routed through the API Gateway to the Path Exploration Service, which retrieves and processes path data from the Persistence Store.

This increment ensures that the internal logic for path retrieval, filtering, and presentation is correct and stable before introducing contextual enrichments such as maps and weather information.

Fourth Incremental Test This increment validates the core functionalities related to trip management, focusing on trip planning and lifecycle handling. The tested features include the creation and storage of planned trips, the management of trip state transitions from planned to ongoing and completed, and the visualization of completed trips in the user's trip history.

At the architectural level, the client allows the user to select a path and initiate trip-related actions, while the API Gateway routes the requests to the appropriate back-end services. The User Management Component ensures that only authenticated users can access these functionalities. The Path Exploration Service provides the selected path information required for planning, and the Trip Management Service manages trip persistence, state transitions, and history retrieval through interactions with the Persistence Store.

This increment establishes a stable foundation for advanced trip-related features, such as statistics computation, tracking, and reporting, which are introduced in subsequent increments.

Fifth Incremental Test This increment focuses on validating the functionalities related to trip statistics and trip history management. The tested features include the computation and visualization of performance statistics for completed trips, such as distance, duration, and average speed, as well as the soft deletion of trips from the user's history.

The client enables the user to access statistics and request trip deletion, while the API Gateway routes these requests to the Trip Management Service. This service is responsible for computing statistics based on persisted trip data, enforcing consistency by excluding soft-deleted trips from aggregated views, and updating the Persistence Store accordingly.

By completing this increment, the system ensures that historical trip data is correctly managed and that performance analytics remain accurate and consistent.

Sixth Incremental Test This increment validates the tracking pipeline responsible for managing sensor and GPS data generated by the user's device during a trip. The tested functionalities include the real-time ingestion of tracking data, its asynchronous processing, and the correct recording of trips based on the collected sensor information.

Architecturally, the User Device sends tracking data to the Tracking Ingestion Endpoint, which immediately publishes tracking events to the Event Bus without performing processing or persistence. The Tracking Processing Worker consumes these events, cleans and normalizes the data, and persists the processed tracking information in the Persistence Store. In parallel, the Trip Management Service updates the state of the trip (e.g., from ongoing to completed) based on tracking-related events and makes the recorded data available for subsequent statistics and reporting.

This increment ensures that continuous data streams are handled efficiently without impacting synchronous system operations, establishing the foundation for automatic reporting and alert evaluation.

Seventh Incremental Test This increment focuses on validating the reporting functionalities that allow the system to collect, manage, and publish information about bike paths. The tested features include the automatic generation of report drafts based on recorded tracking data, the validation of automatically generated reports by the user, the manual creation of reports, and the publication of validated reports.

From an architectural perspective, the Tracking Pipeline provides the processed tracking data required for automatic report generation. The Auto Report Generator Worker creates report drafts asynchronously, while the Reporting component manages the report lifecycle, including validation and publication. Once a report is published, the Path Exploration Service is updated with the new path-related information (e.g., obstacles or hazards), ensuring that the inventory reflects the most recent and reliable data.

This increment ensures that reporting workflows are correctly integrated with both tracking and path exploration, enabling accurate enrichment of path information and supporting subsequent alert evaluation.

Eighth Incremental Test This increment validates the alert management functionalities related to obstacle detection and contextual risk evaluation. The tested features include the evaluation of newly published reports, the assessment of weather conditions impacting planned trips, and the delivery of alert notifications to affected users.

At the architectural level, the Reporting component publishes events when new reports are validated and published. These events, together with contextual updates such as weather information, are consumed by the Alert Evaluation Worker, which evaluates severity thresholds and determines whether planned trips are impacted. The Trip Management Service provides information about users' planned trips and their associated paths, allowing alerts to be targeted only to relevant users. When critical conditions are detected, the alert management logic triggers notifications in an event-driven manner, without relying on continuous polling.

This increment ensures that alert evaluation and notification mechanisms are reactive, decoupled, and correctly integrated with reporting and trip management, completing the system's support for safety-critical user notifications.

5.3 System Testing

The following validation and verification procedures are executed after all platform functionalities are verified:

- **Functional Testing:** validates the system against the Requirement Analysis and Specification Document to ensure all business threads operate as intended.

- Performance Testing: evaluates non-functional requirements to identify implementation inefficiencies that could hinder system speed or stability.
- Usability Testing: focuses on accessibility and user interaction, ensuring the BBP platform remains intuitive and user-friendly for its broad audience.
- Load Testing: simulates expected traffic volumes to verify that the system maintains efficiency and stability under concurrent user activity.
- Stress Testing: pushes the system beyond its normal operational limits to identify breaking points and evaluate how it recovers from extreme conditions.

6 Effort Spent

The following table displays the effort spent in hours of both group members on the sections of the document. The division is only approximative and each section still required the collaboration of both members.

	Section 1	Section 2	Section 3	Section 4	Section 5
Vokrri Fabio	1	24	0	1	12
Soufien Sadgal	0	28	0	2	12

Table 7: Effort spent in hours.

7 References

The following softwares were used to produce the RASD and DD of the Best Bike Paths project:

- All diagrams were made using the Draw.io 29.2.9 software.
- All mock-ups were made using the Figma 125.11.6 software.
- Overleaf web application for document writing and rendering.

Also, generative AI models were used to refine sentences and elevate writing style inside the DD and RASD documents and to check how to model some aspects of the BBP software in UML. Also LLMs were used to overcome some difficulties with $\text{\LaTeX}$ itself, as it sometimes produces cryptic error messages only OCaml programmers can understand. More specifically, the Gemini Pro and ChatGPT models were used, but all the thinking process and the final design and implementation decisions were made by humans. All the outputs of these tools were verified, further refined and integrated with the documents.